
Generating Computational Cognitive Models using Large Language Models

Milena Rmus †
Helmholtz Munich
milena.rmus@helmholtz-munich.de

Akshay K. Jagadish †
Princeton University
akshay.jagadish@princeton.edu

Marvin Mathony
Helmholtz Munich
marvin.mathony@helmholtz-munich.de

Tobias Ludwig
Tübingen University
tobias.ludwig@uni-tuebingen.de

Eric Schulz
Helmholtz Munich
eric.schulz@helmholtz-munich.de

† Milena Rmus and Akshay K. Jagadish contributed equally.

Abstract

Computational cognitive models, which formalize theories of cognition, enable researchers to quantify cognitive processes and arbitrate between competing theories by fitting models to behavioral data. Traditionally, these models are handcrafted, which requires significant domain knowledge, coding expertise, and time investment. However, recent advances in machine learning offer solutions to these challenges. In particular, Large Language Models (LLMs) have demonstrated remarkable capabilities for in-context pattern recognition, leveraging knowledge from diverse domains to solve complex problems, and generating executable code that can be used to facilitate the generation of cognitive models. Building on this potential, we introduce a pipeline for Guided generation of Computational Cognitive Models (GeCCo). Given task instructions, participant data, and a template function, GeCCo prompts an LLM to propose candidate models, fits proposals to held-out data, and iteratively refines them based on feedback constructed from their predictive performance. We benchmark this approach across four different cognitive domains – decision making, learning, planning, and memory – using three open-source LLMs, spanning different model sizes, capacities, and families. On four human behavioral data sets, the LLM generated models consistently matched or outperformed the best domain-specific models from the cognitive science literature. To validate these findings, we performed control experiments that investigated (1) the contribution of the different LLM features (model size, model family, capacities); (2) the causal role of different prompt components; (3) the effect of data contamination; (4) the ability to recover ground truth models from simulated data; and (5) the total explainable variance in human behavior captured by LLM-generated models. Taken together, our results suggest that LLMs can rapidly generate cognitive models with conceptually plausible models that rival – or even surpass – the best models from the literature across diverse task domains. The code for GeCCo is available at <https://github.com/MilenaCCNlab/gecco.git>

1 Introduction

Across the sciences, computational models provide a formal framework for understanding natural phenomena. In cognitive science, these models ground theories about the cognitive processes that underlie human behavior. In practice, researchers hand-craft these models, fit them to behavioral data, and iteratively refine them until they capture behavior sufficiently well (Polk and Seifert, 2002). However, handcrafting cognitive models that accurately explain behavior can be time-consuming (Musslick et al., 2024b,a). It requires trained researchers to conduct lengthy literature reviews, formulate cognitively plausible theories, and implement computationally feasible algorithms to evaluate these theories. While assumptions are necessary to constrain any model, researchers’ own theoretical commitments and expertise may unintentionally narrow the range of models they consider (Krefeld-Schwalb et al., 2022; Taatgen et al., 2016), potentially missing better explanations of the data (Frischkorn and Schubert, 2018; Addyman and French, 2012).

Recent progress in Large Language Models (LLMs) offers a practical route to addressing these challenges, tackling both efficiency and biases, while broadening the space of possible models (Wang et al., 2023; Binz et al., 2023; Musslick et al., 2024a; Li et al., 2024a). Specifically, LLMs exhibit at least three abilities that can facilitate the development of a flexible, domain-general framework for generating cognitive models. First, LLMs can process behavioral data formatted in natural language along with the corresponding task description (Schubert et al., 2024; Jagadish et al., 2024), providing them with the flexibility to handle diverse task domains with varying levels of complexity (Binz et al., 2024). Second, they can identify patterns in complex problems using domain knowledge in-context, and generate hypotheses about the data-generating process (Xiao et al., 2024). Third, their ability to synthesize highly accurate programs (Austin et al., 2021; Perez et al., 2021) lends itself nicely to cognitive model generation and subsequent evaluation. Importantly, these three abilities have already shown promise in statistical modeling, where LLMs have been used to generate and evaluate models represented as probabilistic programs (Li et al., 2024a).

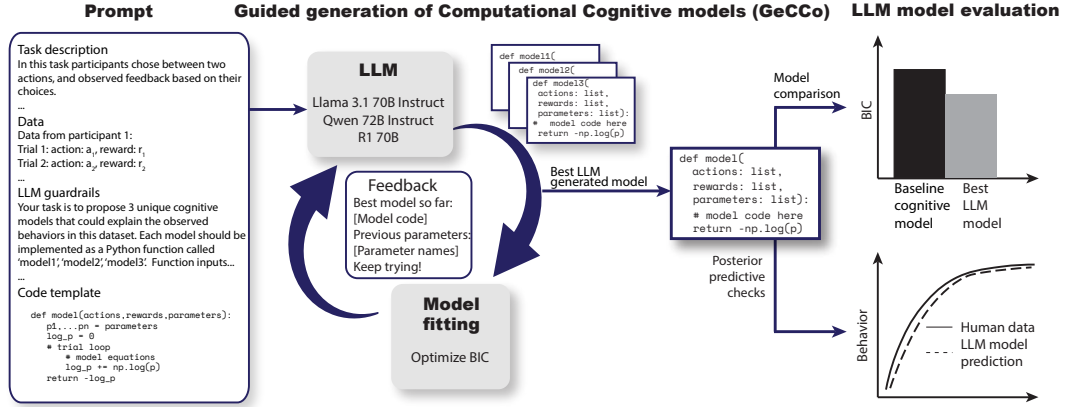


Figure 1: Schematic of GeCCo: We prompt the LLM with a task description, participant data, guardrails to constrain the format of LLM responses, and the code template to generate cognitive models that offer different explanations of the underlying data as Python functions. Model generation evolves over 10 sampling iterations. During each iteration, three LLM-generated models are fitted offline to the held out data (not included in the prompt), and the fitness metric Bayesian Information Criterion (BIC; Watanabe 2013) is used to provide feedback to the LLM on the subsequent iteration. The best model across all 10 sampling iterations is used for evaluation. The LLM-generated models are evaluated by 1) fitting them to behavioral data and comparing the model fit to that of the baseline cognitive model (e.g. the best performing model from the literature) using BIC, and 2) running posterior predictive checks - i.e. simulating the models and comparing simulated to ground truth data - to further verify their validity. For full prompts, see the Appendix B.

In this work, we leveraged these LLM abilities to generate hypotheses about the cognitive processes underlying human behavior. We developed a novel pipeline for Guided generation of Computational Cognitive Models (GeCCo) that helps synthesize cognitive models as Python functions using LLMs, and iteratively refines them based on feedback on their predictive performance. We evaluated

this approach across four different cognitive domains: decision making, learning, planning, and memory, using three open-source LLMs, with different features including model sizes, capacities, and families. Specifically, we first compared LLM-generated models with the best domain-specific model from the literature in terms of predictive performance and then validated the best performing LLM model using posterior predictive checks. Furthermore, we conducted control experiments to gain a mechanistic understanding of this pipeline. These included: (1) linear mixed-effects models to isolate the contribution of different features of the base LLM; (2) LLM prompt ablations to test the causal role of instruction, data, iterative feedback, and template components; (3) LogProber (Yax et al., 2024) estimation of data contamination in prompts; (4) simulations to assess recovery of ground truth models; and (5) a comparison with a foundation model of cognitive science (CENTAUR; Binz et al. 2024) to approximate the total explainable variance captured in human behavior. Across all domains, we found that LLM-generated models matched or surpassed the best domain-specific model from the literature, and captured as much explainable variance as CENTAUR, while remaining interpretable. The results from the control experiments revealed that the iterative feedback component in the prompt and the reasoning ability of the LLMs are the two main drivers of this performance, with no traces of contamination in the prompt. Together, these results position GeCCo as a practical, scalable approach for rapid cognitive-model generation.

2 Methods

Guided generation of Computational Cognitive Models (GeCCo). We implemented GeCCo, a structured guided sampling pipeline for generating cognitive models with LLMs. Each pipeline run consisted of 10 sampling iterations, repeated across 5 independent runs per domain to assess stability. On each iteration, the base LLM was prompted using a fixed prompt structure (see Appendix 6) that included 1) a natural language task description, 2) behavioral data from a subset of participants (prompt data), 3) instructions and guardrails (e.g., write a Python function with a specific name and defined inputs/outputs), 4) a domain-specific template model used primarily for syntactic guidance, and 5) feedback (included from iteration 2 onward). Using this prompt, the LLM generated three distinct cognitive models with non-overlapping parameter sets. These models were parsed from the LLM response, converted into executable Python functions (e.g., via `exec(function_string)`), and their parameter names were extracted. Each model was then fit to a second, held-out validation dataset (not shown in the prompt) using the `minimize` function from the SciPy optimization library (Virtanen et al., 2020), initialized from 20 random starting points to avoid local minima. Model fit was evaluated using the Bayesian Information Criterion (BIC), which was used to identify the current best-performing model. This best model, and a list of previously used parameter names were then included in the next prompt to guide model generation and avoid duplicates. If model generation failed (e.g., due to syntax or runtime errors), the iteration was automatically restarted. After completing all iterations in a run, the best LLM-generated model compared against competing cognitive models by evaluating on a third, held-out test set. See Figure 1 for an overview of the pipeline and Appendix B for exact prompts that were used in each of the experiments.

LLM specifications. We used three open-source LLMs: Llama-3.1-Instruct-70B (Llama; Meta Platforms 2024, DeepSeek-R1-Distill-Llama-3.1-70B (R1; Guo et al. 2025), and Qwen2.5-72B-Instruct (Qwen; Yang et al. 2024). We chose these models as they span different sizes (70B, 72B), capabilities (with/without reasoning), and families (Qwen, Llama, and Deepseek). Importantly, all of our tests were performed in-context. We set the temperature to 0.2 for Llama models, 0.15 for Qwen and 0.1 for R1 to encourage some exploration when generating models. In practice, our approach takes a maximum of 8 hours per task domain on four Nvidia A100s with 40GB memory each.

Evaluation. We applied GeCCo to four canonical cognitive domain paradigms: decision making, learning, planning, and memory. These paradigms include well-established baseline models that are easy to fit and support systematic increases in complexity, both in parameter count and in the cognitive processes they formalize. We began with a simple decision-making task, progressed to sequential learning with trial dependencies, then examined a planning task involving multi-step reasoning, and finally used a task that examines interactions between working memory and reinforcement learning. Evaluating LLM-generated models on human behavioral datasets from these paradigms, where data are noisy and no single model fully captures the underlying processes, is essential, as it reflects the real-world settings in which cognitive scientists would use our approach.

Metrics. We evaluated the performance of the GeCCo generated model using the Bayesian Information Criterion (BIC; Watanabe 2013), which balances goodness of fit and model complexity by penalizing the number of parameters. Because BIC is a relative metric, when models achieve similar fit, it favors the less complex (fewer parameter) model. The pipeline is agnostic to the model quality measure; for example, in Appendix I, we report results using the Akaike Information Criterion (AIC) as a metric. In the last stage of the evaluation, we additionally computed the exceedance probability (EXP; Stephan et al. 2009), which quantifies the posterior probability that one model explains the data better than its competitors. EXP expresses, in probabilistic terms, how confident we are that a given model provides the most prevalent explanation of the observed behavior.

3 Experiment 1: Decision making

Task. To understand the mechanisms underlying human decision making, researchers have commonly resorted to multi-feature decision making tasks (Bogacz et al., 2006). In such tasks, participants typically choose between multiple options, each defined by a distinct set of features. We considered the paradigm from Hilbig and Moshagen (2014), where participants chose between two options that are defined by four feature values and their respective validities (Fig. 2A; see Appendix E for more details).

Baseline cognitive model. The winning model in the Hilbig and Moshagen (2014) study was a probabilistic weighted additive model (pWADD), which combines feature values and inferred feature weights (instead of feature validities) to predict human choices. Critically, the inferred weights can match or completely differ from the feature validity provided by the experimenter; see Appendix F.1.

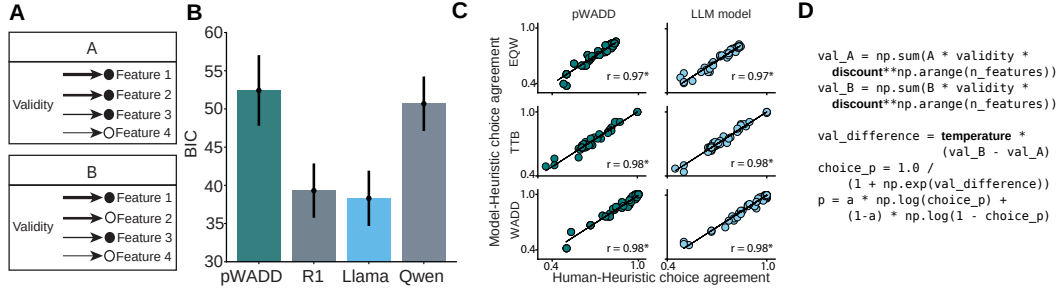


Figure 2: Experiment 1: Decision Making. A) Schematic of the decision task from Hilbig and Moshagen (2014), where participants were asked to choose between two options based on four binary features and their validities. Arrow thickness indicates validity value (i.e., thicker arrows mean higher validity). B) Model fit comparison: LLM-generated models from R1 and Llama outperformed the best literature model. C) Posterior predictive checks showed that proportions of choices accounted for by the canonical heuristics (Equal Weighting, Take The Best and Weighted Additive Heuristics) closely aligned between human data and respective LLM and pWADD model predictions. D) Code of the best LLM-generated model (Llama), which can arbitrate between three canonical heuristics mentioned above via a discount factor.

Results. We found that both R1 and Llama produced a model that outperformed the best literature model (Llama $BIC_{M \pm SEM}$ vs. pWADD $BIC_{M \pm SEM}$: 39.40 ± 4.54 vs. 51.97 ± 4.52 ; t-test (pWADD > Llama BIC: $t(52) = 41.8$, $p < .001$; see Figure 2B). The best LLM-generated model (by Llama) weighs the option features by the respective feature validities and a discount factor. The strength of this model is that it can arbitrate between Take the Best, Weighted Additive, and Equal Weighting decision making strategies with a single parameter; see Figure 2D. Although the pWADD model can do the same using its inferred feature weights, it requires four parameters instead of one parameter used by the LLM-generated model. To validate the LLM-generated model, we performed posterior predictive checks - which typically includes simulating data based on the cognitive model, and checking whether the simulated data agrees with the human data (for details, refer to Appendix H). We found that the data simulated from the best LLM-generated model aligned closely with human choices; see Figure 2C.

Scientific Insight. The parsimonious way in which the LLM-generated model selects between heuristics aligns with the continuum-of-models view, which interprets heuristics as Bayesian inference under extreme priors (Parpart et al., 2018). In fact, the discount term in the LLM-generated model parallels the prior-strength parameter in the constraint-optimal regression and half-ridge formulations from Parpart et al. (2018), both of which enable smooth modulation of posterior feature weights—and therefore the decision rule. Importantly, the LLM-generated model achieves this flexibility without adopting a Bayesian framework.

4 Experiment 2: Learning

Task. Multi-armed bandit tasks are frequently used in the study of feedback-driven learning (Frank et al., 2004; Pessiglione et al., 2006; Wang et al., 2016). Generally, in a bandit task, an agent chooses between N arms, often with a predefined reward contingency associated with each arm. The agent receives feedback for their chosen action and, over a sequence of trials, learns to adjust action selection in a way that maximizes positive outcomes. For analysis, we considered the two-armed bandit task dataset from Chambon et al. (2020). In addition to the classic condition (partial feedback), where only rewards for chosen options are observed (see Appendix G and Fig. 12 for results), the authors include a full-feedback condition, where rewards for both unchosen and chosen options are observable; see Appendix E for details.

Baseline cognitive model. The winning model from Chambon et al. (2020) is a variant of the classic Rescorla-Wagner model that is commonly used to study learning dynamics in bandit tasks. This model ($RW^{4\alpha}$) contains four learning rates: learning rates for positive and negative prediction errors for chosen actions, and the same for unchosen actions. In addition to using a delta rule for action-value updating, the RW model employs a softmax policy for action selection, with an inverse temperature parameter that transforms values into probabilities and a perseveration parameter that increases the weight of previously chosen actions. For full model details, see Appendix F.2.

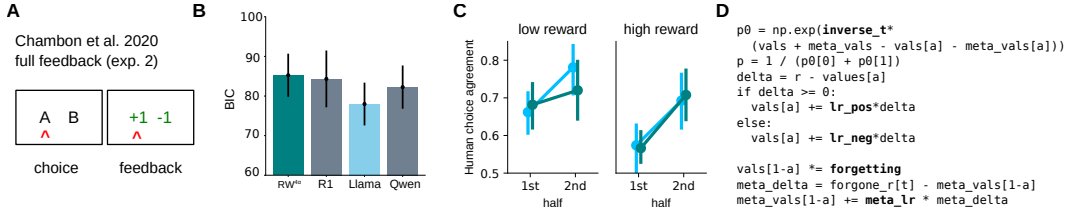


Figure 3: Experiment 2: Learning. A) Schematic of the learning task from Chambon et al. 2020, where participants chose between two options and received feedback for both, but only got the reward for the chosen option. B) Model fit comparison: LLM-generated models from Llama on average fit better than $RW^{4\alpha}$. C) Posterior predictive checks showing close alignment between human data and predictions of the best LLM model in both low and high reward blocks. D) Code of the best LLM-generated model (Llama), which displays asymmetric learning rates along with forgetting of values, and a dedicated fictive trace for counterfactual outcomes.

Results. We found that Llama ($BIC_{M \pm SEM} = 77.97 \pm 5.40$) produced a model that performed slightly better than $RW^{4\alpha}$ ($BIC_{M \pm SEM} = 85.24 \pm 7.95$); see Figure 3B. However, we found that, in terms of the exceedance probability (EXP), the LLM model (0.97) scored significantly higher compared to the best model reported in the literature (0.03). In addition to differentiating learning rates based on the valence of the prediction error, the best LLM-generated model included two key components: a forgetting parameter that decays the value of the unchosen action (potentially capturing working memory limits), and a separate meta-value function (and learning rate) that learns uniquely from forgone outcomes (capturing regret/relief signals in a dedicated fictive trace); see Figure 3D. For posterior predictive checks, we assessed the agreement between model-predicted choices and actual human choices separately for high- and low-reward blocks. The LLM-generated model matched human behavior as well as $RW^{4\alpha}$ (Figure 3C).

Scientific Insight. The GeCCo-generated model separates two sets of Q-values: one updated from chosen outcomes and another counterfactual set updated from forgone outcomes. Whereas

standard models often merge these via asymmetric learning rates, GeCCo keeps them distinct, raising questions about how experienced and counterfactual values are weighted in decision-making. The model also applies forgetting to unchosen actions, creating temporal asymmetry in value retention and eliminating the need for an explicit choice stickiness parameter—often included mainly for model fit rather than theory. This illustrates how GeCCo can inspire rethinking and improving cognitive models.

5 Experiment 3: Planning

Task. A widely used paradigm in the planning literature is the two-stage task introduced by Daw et al. (2011). The goal of this task is to investigate whether people learn action-reward contingencies using habitual, reward-driven learning (referred to as model-free), or they make decisions based on the underlying state-transition structure that they estimate from experience (model-based). We considered a planning dataset from Feher da Silva and Hare (2020), which replicated the original study (Daw et al., 2011); see Appendix E for details of the experiment.

Baseline cognitive model. For baseline, we considered the Hybrid model from Daw et al. (2011) – the best performing model from the literature, which effectively combines model-free action values from a SARSA(λ) algorithm with model-based values (updated based on the transition probabilities), weighted by a free parameter. For detailed equations, see Appendix F.3.

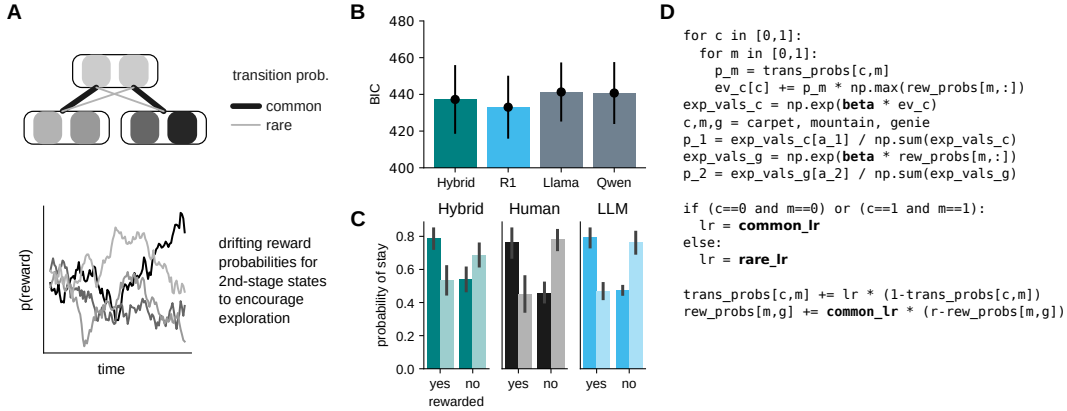


Figure 4: Experiment 3: Planning. A) Schematic of the planning task from Feher da Silva and Hare (2020), where participants took two steps in a stochastic environment with common and rare transitions and fluctuating rewards. B) Model fit comparison: the model generated by R1 had a lower average BIC score compared to the Hybrid model. C) Posterior predictive checks showing the same pattern across humans, literature model and R1-generated model of repetition of common (dark) and rare (bright) transitions depending on if the previous action was rewarded. D) Code of the best LLM-generated model (R1), which uses separate learning rates for common and rare transitions and inverse temperature for exploration – omitting discounting of rewards.

Results. As shown in Figure 4B, R1 generated the best overall model that achieved a lower mean BIC (432.47 ± 19.31) than the hybrid (437.38 ± 18.72). While the BIC difference was not significant (t-test (Hybrid > R1 BIC): $t(15) = 0.61$; $p = 0.27$), the exceedance probability (EXP) favored the R1 model (R1 EXP = 0.85 versus Hybrid EXP = 0.15). Posterior predictive checks revealed that the LLM-generated model, like humans, made action selections consistent with a more model-based strategy; see Figure 4C. The code generated by R1 describes a version of the transition dependent learning rate model that uses two separate learning rates for common and rare transitions, respectively (similar to one of the candidate models reported by Feher da Silva and Hare (2020)). Furthermore, it learns the transition probabilities, indicating that it is not model-free and behaves similarly to humans. It also incorporates an inverse temperature parameter to regulate exploration, but notably omits a discounting parameter, which is typically included in multi-step planning tasks to devalue distant rewards. Drawing on reinforcement learning literature, the model implements value iteration (Bellmann, 1958), taking the maximum over future value estimates (line 4 of the code) to update the expected value for the first-stage actions; see Figure 4D.

Scientific Insight. The GeCCo-generated model offers a parsimonious account of planning, showing how behavior based on learned transition structures can emerge from general principles of transition learning and value updating, without explicitly separating model-based and model-free systems. This provides an alternative to traditional reinforcement learning frameworks that rely on multiple canonical components and aligns with recent calls to move beyond the strict MB/MF dichotomy (Collins and Cockburn, 2020).

6 Experiment 4: Working memory

Task. The reinforcement learning–working memory (RL-WM) paradigm was designed to disentangle reinforcement learning (RL) and working memory (WM) contributions to the learning process (Collins and Frank, 2012). In this paradigm, participants learn a varying number of stimulus–action (S-A) associations per block via feedback, receiving a reward for correct responses to individual stimuli. Varying the cognitive load targets the WM capacity by increasing the number of associations and the delay between stimulus repetitions. We used data from the version reported in Rmus et al. (2023) with two set sizes: three (low) and six (high) S-As ; see Appendix E for details.

Baseline cognitive model. As a baseline, we used the RL-WM model, which consists of independent but interacting reinforcement learning (RL) and working memory (WM) modules to isolate their respective contributions to learning. The RL module captures incremental, capacity-unlimited learning, whereas the WM module supports one-shot learning with perfect retention from the previous trial but is capacity-limited and subject to decay. For detailed equations, see Appendix F.4.

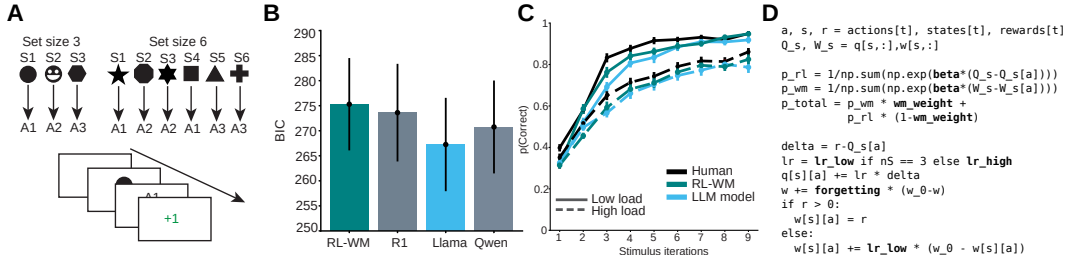


Figure 5: Experiment 4: Working Memory. A) Schematic of the reinforcement learning - working memory task from Rmus et al. (2023), where participants learned a varying number of state-action associations. B) Model fit comparison: Llama-generated models outperformed the best literature model. C) Posterior predictive checks showing close alignment between human data and predictions of the best LLM model. D) Code of the best LLM-generated model (Llama), which distinguishes between fast learning under low cognitive load (working memory) and slower learning under high cognitive load.

Results. We found that Llama ($BIC_{M \pm SEM} = 267.25 \pm 9.31$) produced a model that, on average, had a lower average BIC score than the RL-WM model ($BIC_{M \pm SEM} = 275.2 \pm 9.21$; t-test (RL-WM > Llama BIC): $t(24) = 14.2$; $p < .01$); Figure 5B. In terms of exceedance probability (EXP), the Llama-generated model (0.99) scored higher than RL-WM (0.01). The LLM-generated cognitive model implemented differences in learning based on low vs. high cognitive load, but did so in the RL module, rather than using the cognitive load-dependent WM weight (as is the case in the original RL-WM model); Figure 5D. Posterior predictive checks revealed that the LLM-generated model captured key patterns in human behavior reasonably well: faster, asymptotic learning in the low cognitive load condition typically associated with WM, and incremental, RL-like learning in the high cognitive load condition; Figure 5C.

Scientific Insight. The LLM-generated model captured behavioral patterns and fit the human data well, even though the cognitive load manipulation primarily affected the RL rather than the WM component (Collins and Frank, 2012). This raises questions about where cognitive load effects arise and challenges the assumption that RL processes are relatively insensitive to load. Notably, GeCCo’s proposal aligns with evidence that RPE signals—central to reinforcement learning—are blunted

under low cognitive load (Collins et al., 2017), suggesting that the RL system may adjust learning rates based on WM load.

7 Control experiments

Contribution of different model features. To examine the importance of LLM features—reasoning ability, base model family, and model size—we fit a mixed-effects regression predicting the BIC of the best LLM-generated models from these three factors, with participant-specific random intercepts. Reasoning ability (as in R1) yielded slightly better models ($\beta_{M \pm \text{SEM}} = -0.266 \pm 1.029$, $p = 0.07$), though the effect was not significant. In contrast, Llama outperformed Qwen as a base model ($\beta_{M \pm \text{SEM}} = 5.624 \pm 1.029$, $p = 0.03$), despite Qwen having more parameters—likely due to differences in training data and/or training protocols.

Causal role of different prompt components. We conducted an ablation study to assess the causal contribution of each prompt component—feedback, code template, task description, and participant data sequences—by systematically ablating each element and rerunning the model generation pipeline (see Figure 6 for the full prompt structure). To estimate their joint influence, we fit a linear mixed-effects regression predicting post-ablation BIC scores from binary-coded prompt components, with random intercepts per participant. Feedback was the strongest predictor of model performance ($\beta_{M \pm \text{SEM}} = -17.040 \pm 2.287$, $p < 0.05$), followed by participant data ($\beta_{M \pm \text{SEM}} = -12.836 \pm 2.287$, $p < 0.05$) and task description ($\beta_{M \pm \text{SEM}} = -9.834 \pm 2.287$, $p < 0.05$). The code template had the weakest effect ($\beta_{M \pm \text{SEM}} = -9.359 \pm 2.287$, $p < 0.05$; see Figure 9).

Data contamination analysis. We checked for data leakage in the prompts used to generate cognitive models using the LogProber method (Yax et al., 2024). Upon applying it to Llama (the base model that generated the best-performing cognitive models most frequently), we found no evidence of data contamination, with the acceleration term substantially below the threshold of 1 recommended by Yax et al. (2024) for the four human experiments. Specifically, it was 0.0171 for the prompts used for decision-making, 0.1147 (full feedback) and 0.1409 (partial feedback) for learning, 0.1718 for planning, and 0.6481 for working memory; see Figure 10 for details.

Recovering ground truth models in synthetic datasets. Synthetic datasets generated through model simulations provide full control over task behavior and access to ground truth, making them ideal for benchmarking. As a sanity check, we evaluated our approach on synthetic data from two domains: learning and decision making. In both cases, the LLM inferred models that closely matched the true data-generating processes, supporting the validity of our method. In decision making, the LLM recovered key heuristics: single-feature focus for Take the Best and cross-feature comparisons for Tallying. In learning, it correctly identified the Rescorla-Wagner (RW) model with two learning rates when fitted to corresponding data, and for data from an RW model with stickiness, it produced a similar value-decay formulation. See Appendix C, Appendix D, Figure 7, and Figure 8 for details.

Explainable variance captured in human behavior. To assess explainable variance in human behavior captured by LLM-generated models, we compared its negative log-likelihood to that of CENTAUR (Binz et al., 2024) - a foundation model trained to predict human behavior across multiple cognitive domains - on held-out data. As CENTAUR outperforms domain-specific models, it serves as a proxy for maximum explainable variance. The LLM-generated models matched or exceeded its performance across domains (see Table 1), indicating that they captured most of the explainable variance while remaining interpretable.

Table 1: Mean negative log-likelihoods ($M \pm \text{SE}$) and paired t -test results of CENTAUR vs. LLM-generated cognitive model across four task domains.

DOMAIN	CENTAUR ($M \pm \text{SE}$)	LLM ($M \pm \text{SE}$)	t	p
Decision-making	15.41 $\pm$ 1.99	15.08 $\pm$ 2.29	0.27	0.78
Learning	53.58 $\pm$ 3.31	26.17 $\pm$ 2.79	18.83	< 0.001
Planning	206.00 $\pm$ 11.14	214.10 $\pm$ 10.59	-1.42	0.18
Memory	127.44 $\pm$ 4.68	120.67 $\pm$ 4.68	5.75	< 0.001

8 Discussion

We developed GeCCo, a novel pipeline that uses open-source LLMs to generate cognitive models as Python functions and iteratively refine them via feedback. We applied GeCCo to paradigms from four cognitive domains and found that LLM-generated models matched or outperformed domain-specific baselines in all cases. Control experiments showed that 1) the base LLM model family was an important determinant of performance, 2) all prompt components contributed meaningfully, but iterative feedback was causally the most important, and 3) No significant data leakage occurred in task prompts. We also demonstrated that GeCCo recovered ground truth models from simulated data and produced interpretable models that outperformed CENTAUR in predictive power, indicating that LLM-generated models captured most of the explainable variance in human behavior.

8.1 Related work

Model discovery with LLMs. The long-standing goal of automated model discovery holds promise of accelerating and democratizing scientific discovery by reducing dependence on prior expertise. Until recently, such efforts relied on domain-specific languages and hand-crafted search algorithms to explore predefined model spaces (Kemp and Tenenbaum, 2008; Lloyd et al., 2014; Musslick et al., 2024a; Gulwani, 2011; Steinruecken et al., 2019; Hewson et al., 2023). Recent advances in LLMs have overcome many of these limitations. Researchers have used LLMs to automate statistical model discovery (Li et al., 2024a), solve classic ML tasks (Xiao et al., 2024), support hypothesis generation and refinement (Zhou et al., 2024), propose valid domain rules (e.g., chemistry) (Zheng et al., 2023b), and even automate the entire scientific pipeline, from experiment design to peer review, in machine learning (Lu et al., 2024). In cognitive modeling, LLMs have been used to translate human strategy descriptions into executable code (Xie et al., 2024), and to generate cognitive models from task descriptions and data using evolutionary search (Castro et al., 2025). Our approach complements these concurrent efforts by offering a lightweight open-source method that relies purely on in-context learning, enabling much faster convergence (hours instead of days).

Code writing abilities of LLMs. The promise of LLMs in model search lies not only in their broad knowledge but also in their ability to interpret natural language and synthesize executable code. Using LLMs to generate models in general-purpose languages like Python provides a path beyond handcrafted, domain-specific languages for automating model discovery (Austin et al., 2021; Ni et al., 2024). Prior work has shown that LLMs excel at mathematical and programming tasks (Austin et al., 2021; Ni et al., 2024; Perez et al., 2021) and can generate mathematical functions and probabilistic Python programs that model input data (Li et al., 2024a; Xiao et al., 2024; Zheng et al., 2023b). More recently, LLMs have advanced program induction, with successes in domains such as abstract reasoning (Li et al., 2024b) and graphical function induction (Li and Ellis, 2024).

8.2 Discussion and Limitations

A key advantage of our approach is that all results were obtained purely in-context using open-source LLMs, without any fine-tuning. This drastically reduces the barrier to entry, as researchers could use our pipeline off-the-shelf to generate alternative models for their behavioral data. GeCCo implements a hybrid optimization loop, where the LLM generates candidate models that are fit to the data offline via traditional optimization tools. This ensures that model selection remains data-driven, with the LLM acting as a proposal engine rather than an unverified model generator. Finally, our pipeline generalizes well across diverse task domains, making it adaptable to other researchers. These advantages suggest that our approach can help cognitive scientists explore a broader model space more efficiently.

Despite its success, our approach has some limitations. Our approach uses a simple domain-specific model as a template in the prompt. Although this could bias the LLM toward a particular model class, an ablation analysis showed that removing the template had the smallest impact on downstream model performance. To further reduce dependency on handcrafted templates, we also generated the template with the LLM itself, which produced models that fit the data just as well (see Appendix J). Currently, our approach uses offline-computed model fit metrics (e.g., BIC) to generate feedback, consisting of the current best-performing model and previously tested parameter combinations. This could be enhanced by incorporating feedback from another LLM based on predefined criteria such as

theoretical plausibility, parsimony, alternative hypotheses, or code quality, which may further reduce the need for human oversight. It is also important to note that the domains examined in this paper do not represent the full scope of cognitive modeling. Future work should extend our approach to other areas such as language processing and perception. Applying GeCCo to higher-dimensional data (e.g., vision or language) and generating models with many parameters may be challenging, but it represents an exciting direction for future research. Furthermore, to make our approach valuable beyond cognitive science, it should be evaluated on more naturalistic datasets that capture human behavior in real-world settings.

Overall, our findings suggest that LLMs can significantly advance computational modeling in cognitive science by democratizing access to complex model discovery and accelerating the pace of research.

Acknowledgements

We thank all reviewers for their constructive and thoughtful feedback. We also thank the authors of Chambon et al. (2020), Rmus et al. (2023), Feher da Silva and Hare (2020), and Hilbig and Moshagen (2014) for making the data from their study available. Furthermore, we thank the members of the “Human-Centered Artificial Intelligence” (HCAI Lab) for their comments, discussions, and support. This work was supported by the Volkswagen Foundation, Princeton AI Lab, Helmholtz Munich and the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy–EXC2064/1–390727645.15/18.

References

- Addyman, C. and French, R. M. (2012). Computational modeling in cognitive science: A manifesto for change. *Topics in Cognitive Science*, 4(3):332–341.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. (2021). Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.
- Balcarras, M., Ardid, S., Kaping, D., Everling, S., and Womelsdorf, T. (2016). Attentional selection can be predicted by reinforcement learning of task-relevant stimulus features weighted by value-independent stickiness. *Journal of cognitive neuroscience*, 28(2):333–349.
- Bellmann, R. (1958). Dynamic programming and stochastic control processes. *Information and Control*.
- Binz, M., Akata, E., Bethge, M., Brändle, F., Callaway, F., Coda-Forno, J., Dayan, P., Demircan, C., Eckstein, M. K., Éltető, N., et al. (2024). Centaur: a foundation model of human cognition. *arXiv preprint arXiv:2410.20268*.
- Binz, M., Alaniz, S., Roskies, A., Aczel, B., Bergstrom, C. T., Allen, C., Schad, D., Wulff, D., West, J. D., Zhang, Q., et al. (2023). How should the advent of large language models affect the practice of science? *arXiv preprint arXiv:2312.03759*.
- Bogacz, R., Brown, E., Moehlis, J., Holmes, P., and Cohen, J. D. (2006). The physics of optimal decision making: a formal analysis of models of performance in two-alternative forced-choice tasks. *Psychological review*, 113(4):700.
- Castro, P. S., Tomasev, N., Anand, A., Sharma, N., Mohanta, R., Dev, A., Perlin, K., Jain, S., Levin, K., Éltető, N., et al. (2025). Discovering symbolic cognitive models from human and animal behavior. *bioRxiv*, pages 2025–02.
- Chambon, V., Théro, H., Vidal, M., Vandendriessche, H., Haggard, P., and Palminteri, S. (2020). Information about action outcomes differentially affects learning from self-determined versus imposed choices. *Nature Human Behaviour*, 4(10):1067–1079.
- Collins, A. G., Ciullo, B., Frank, M. J., and Badre, D. (2017). Working memory load strengthens reward prediction errors. *Journal of Neuroscience*, 37(16):4332–4342.

- Collins, A. G. and Cockburn, J. (2020). Beyond dichotomies in reinforcement learning. *Nature Reviews Neuroscience*, 21(10):576–586.
- Collins, A. G. and Frank, M. J. (2012). How much of reinforcement learning is working memory, not reinforcement learning? a behavioral, computational, and neurogenetic analysis. *European Journal of Neuroscience*, 35(7):1024–1035.
- Daw, N., Gershman, S., Seymour, B., Dayan, P., and Dolan, R. (2011). Model-Based Influences on Humans’ Choices and Striatal Prediction Errors. *Neuron*, 69(6):1204–1215.
- Diederen, K. M. and Schultz, W. (2015). Scaling prediction errors to reward variability benefits error-driven learning in humans. *Journal of neurophysiology*, 114(3):1628–1640.
- Fehér da Silva, C. and Hare, T. A. (2020). Humans primarily use model-based inference in the two-stage task. *Nature Human Behaviour*, 4(10):1053–1066.
- Frank, M. J., Seeberger, L. C., and O’reilly, R. C. (2004). By carrot or by stick: cognitive reinforcement learning in parkinsonism. *Science*, 306(5703):1940–1943.
- Frischkorn, G. T. and Schubert, A.-L. (2018). Cognitive models in intelligence research: Advantages and recommendations for their application. *Journal of Intelligence*, 6(3):34.
- Gigerenzer, G. and Goldstein, D. G. (1996). Reasoning the fast and frugal way: models of bounded rationality. *Psychological review*, 103(4):650.
- Gulwani, S. (2011). Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Hewson, J. T. S., Strittmatter, Y., Marinescu, I., Williams, C. C., and Musslick, S. (2023). Bayesian machine scientist for model discovery in psychology. In *NeurIPS 2023 AI for Science Workshop*.
- Hilbig, B. E. and Moshagen, M. (2014). Generalized outcome-based strategy classification: Comparing deterministic and probabilistic choice models. *Psychonomic bulletin & review*, 21:1431–1443.
- Jagadish, A. K., Coda-Forno, J., Thalmann, M., Schulz, E., and Binz, M. (2024). Human-like category learning by injecting ecological priors from large language models into neural networks. In *International Conference on Machine Learning (ICML)*.
- Kemp, C. and Tenenbaum, J. B. (2008). The discovery of structural form. *Proceedings of the National Academy of Sciences*, 105(31):10687–10692.
- Krefeld-Schwalb, A., Pachur, T., and Scheibehenne, B. (2022). Structural parameter interdependencies in computational models of cognition. *Psychological Review*, 129(2):313.
- Li, M. Y., Fox, E. B., and Goodman, N. D. (2024a). Automated statistical model discovery with language models. *arXiv preprint arXiv:2402.17879*.
- Li, W.-D., , , and (2024b). Combining induction and transduction for abstract reasoning. *arXiv preprint arXiv:2411.02272*.
- Li, W.-D. and Ellis, K. (2024). Is programming by example solved by llms? *arXiv preprint arXiv:2406.08316*.
- Lloyd, J., Duvenaud, D., Grosse, R., Tenenbaum, J., and Ghahramani, Z. (2014). Automatic construction and natural-language description of nonparametric regression models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 28.
- Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. (2024). The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.
- Meta Platforms, I. (2024). Llama 3.1 70b model.

- Musslick, S., Bartlett, L. K., Chandramouli, S. H., Dubova, M., Gobet, F., Griffiths, T. L., Hullman, J., King, R. D., Kutz, J. N., Lucas, C. G., Mahesh, S., Pestilli, F., Sloman, S. J., and Holmes, W. R. (2024a). Automating the practice of science – opportunities, challenges, and implications.
- Musslick, S., Strittmatter, Y., and Dubova, M. (2024b). Closed-loop scientific discovery in the behavioral sciences.
- Nassar, M. R. and Frank, M. J. (2016). Taming the beast: extracting generalizable knowledge from computational models of cognition. *Current opinion in behavioral sciences*, 11:49–54.
- Ni, A., Yin, P., Zhao, Y., Riddell, M., Feng, T., Shen, R., Yin, S., Liu, Y., Yavuz, S., Xiong, C., et al. (2024). L2ceval: Evaluating language-to-code generation capabilities of large language models. *Transactions of the Association for Computational Linguistics*, 12:1311–1329.
- Parpart, P., Jones, M., and Love, B. C. (2018). Heuristics as bayesian inference under extreme priors. *Cognitive psychology*, 102:127–144.
- Paskewitz, S., Stoddard, J., and Jones, M. (2022). Explaining the return of fear with revised rescorla-wagner models. *Computational Psychiatry*, 6(1):213.
- Perez, L., Ottens, L., and Viswanathan, S. (2021). Automatic code generation using pre-trained language models. *arXiv preprint arXiv:2102.10535*.
- Pessiglione, M., Seymour, B., Flandin, G., Dolan, R. J., and Frith, C. D. (2006). Dopamine-dependent prediction errors underpin reward-seeking behaviour in humans. *Nature*, 442(7106):1042–1045.
- Polk, T. A. and Seifert, C. M. (2002). *Cognitive modeling*. MIT Press.
- Rescorla, R. A. (1972). A theory of pavlovian conditioning: Variations in the effectiveness of reinforcement and nonreinforcement. *Classical conditioning II: Current theory and research/Appleton-Century-Crofts*.
- Rmus, M., He, M., Baribault, B., Walsh, E. G., Festa, E. K., Collins, A. G., and Nassar, M. R. (2023). Age-related differences in prefrontal glutamate are associated with increased working memory decay that gives the appearance of learning deficits. *Elife*, 12:e85243.
- Schubert, J. A., Jagadish, A. K., Binz, M., and Schulz, E. (2024). In-context learning agents are asymmetric belief updaters. *arXiv preprint arXiv:2402.03969*.
- Steinruecken, C., Smith, E., Janz, D., Lloyd, J., and Ghahramani, Z. (2019). The automatic statistician. *Automated machine learning: Methods, systems, challenges*, pages 161–173.
- Stephan, K. E., Penny, W. D., Daunizeau, J., Moran, R. J., and Friston, K. J. (2009). Bayesian model selection for group studies. *Neuroimage*, 46(4):1004–1017.
- Storn, R. and Price, K. (1997). Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11:341–359.
- Taatgen, N. A., van Vugt, M. K., Borst, J. P., and Mehlhorn, K. (2016). Cognitive modeling at iccm: state of the art and future directions. *Topics in Cognitive Science*, 8(1):259–263.
- Tversky, A. and Kahneman, D. (1974). Judgment under uncertainty: Heuristics and biases: Biases in judgments reveal some heuristics of thinking under uncertainty. *science*, 185(4157):1124–1131.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., and SciPy 1.0 Contributors (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272.
- Wang, H., Fu, T., Du, Y., Gao, W., Huang, K., Liu, Z., Chandak, P., Liu, S., Van Katwyk, P., Deac, A., et al. (2023). Scientific discovery in the age of artificial intelligence. *Nature*, 620(7972):47–60.

- Wang, J. X., Kurth-Nelson, Z., Tirumala, D., Soyer, H., Leibo, J. Z., Munos, R., Blundell, C., Kumaran, D., and Botvinick, M. (2016). Learning to reinforcement learn. *arXiv preprint arXiv:1611.05763*.
- Watanabe, S. (2013). A widely applicable bayesian information criterion. *The Journal of Machine Learning Research*, 14(1):867–897.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Wilson, R. C., Bonawitz, E., Costa, V. D., and Ebitz, R. B. (2021). Balancing exploration and exploitation with information and randomization. *Current opinion in behavioral sciences*, 38:49–56.
- Wilson, R. C. and Collins, A. G. (2019). Ten simple rules for the computational modeling of behavioral data. *Elife*, 8:e49547.
- Xiao, T. Z., Bamler, R., Schölkopf, B., and Liu, W. (2024). Verbalized machine learning: Revisiting machine learning with language models. *arXiv preprint arXiv:2406.04344*.
- Xie, H., Xiong, H., and Wilson, R. C. (2024). From strategic narratives to code-like cognitive models: An llm-based approach in a sorting task. In *COLM 2024*.
- Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., et al. (2024). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- Yax, N., Oudeyer, P.-Y., and Palminteri, S. (2024). Assessing contamination in large language models: Introducing the logprober method. *arXiv preprint arXiv:2408.14352*.
- Zheng, H. S., Mishra, S., Chen, X., Cheng, H.-T., Chi, E. H., Le, Q. V., and Zhou, D. (2023a). Take a step back: Evoking reasoning via abstraction in large language models. *arXiv preprint arXiv:2310.06117*.
- Zheng, Y., Koh, H. Y., Ju, J., Nguyen, A. T., May, L. T., Webb, G. I., and Pan, S. (2023b). Large language models for scientific synthesis, inference and explanation. *arXiv preprint arXiv:2310.07984*.
- Zhou, Y., Liu, H., Srivastava, T., Mei, H., and Tan, C. (2024). Hypothesis generation with large language models. In *Proceedings of the 1st Workshop on NLP for Science (NLP4Science)*.

A Prompt structure

Task description

In a 2-armed bandit task, a participant chooses between two actions. The participant's action selection is followed by an outcome: positive (1) if a reward is received, negative (-1) if not.

Data

Here is a task dataset from several participants:

Data from participant XX:
Trial: 0, action: [a0], reward: [r0]
Trial: 1, action: [a1], reward: [r1]
.....

LLM guardrails

Your task is to propose 3 unique cognitive models that could explain the observed behaviors in this dataset.

Make sure all of the model parameters are actually being used. Each model should be implemented as a Python function called: `cognitive\_model1`, `cognitive\_model2`, and `cognitive\_model3`.

Each of the 3 functions should accept the following lists as arguments: actions, rewards, and model parameters. Each function should return the negative log likelihood of observed actions given the model parameters.

Ensure your models have distinct assumptions and parameter sets. Avoid repeating ideas used in previous iterations.

When you write the function, also include the description of each parameter and what it does in the function's meta commented section.

Make sure you write functions that are free of bugs, and can be executed.

Code template

Here is a function template:

```
def cognitive_model(actions, rewards, parameters):  
  
    parameter_1, parameter_2, .... parameter_n = parameters  
    log_likelihood = 0  
    for t in range(len(actions)):  
        ...  
        log_likelihood += np.log(p[actions[t]])  
  
    return -log_likelihood
```

Feedback

Best model you generated so far is:

[Model code]

Parameters you have used so far are:

...

Keep trying!

Figure 6: General structure of the LLM prompt we used across all of the domains. The prompt generally consisted of the paradigm description, data in the text format, Python function specifications (including the name of the function, required arguments and output), code function template, and the feedback (on iterations after the first one) established following offline evaluation of LLM-generated models.

B Prompts

B.1 Decision making

Decision making prompt

This is a multi-attribute decision-making task, where participants have to choose the superior product between two options, labeled A and B. Each option represented a fictitious product and they had to infer which product was superior in terms of quality in every trial. For each option, they were provided with four expert ratings (with 1 representing a positive and 0 representing a negative rating). The four experts differ in their validity. The ratings of experts were given in descending order of their validity (having validities of 0.9, 0.8, 0.7, and 0.6 respectively). Participants selected a product by selecting the corresponding option, i.e. A or B. Here is a task data set from several participants:

Data from participant 1:

Trial 1: Product A ratings: [1 1 1 1]. Product B ratings: [0 0 1 1]. Chosen option: A
Trial 2: Product A ratings: [1 0 0 0]. Product B ratings: [0 0 0 1]. Chosen option: A
Trial 3: Product A ratings: [0 1 1 1]. Product B ratings: [1 1 1 0]. Chosen option: B.

...

Propose three unique cognitive models that could explain the observed behavior in the dataset. Each model should have distinct assumptions and parameters. Avoid repeating ideas used in previous iterations. Think step by step: How do participants use features or expert validities to make their decisions?

Each model should be implemented as a Python function named 'cognitive_model1', 'cognitive_model2', and 'cognitive_model3'. Each function should take following inputs: a list of choices, a list of option A feature lists, a list of option B feature lists and a list of model parameters. Each function should return the negative log likelihood of the observed choices given its parameters.

Clearly define each model parameter and explain its role in the function's commented section. All parameters (except inverse temperature) should have values between 0 and 1. Ensure the equations do not result in nonsense values (e.g., avoid division by zero). Make sure your Python functions are executable and bug-free.

Consider the following code as a function template:

```
def cognitive_model(choices, optionAs, optionBs, validities, params):

    w1, w3, w3, w4, temperature = params
    v1, v2, v3, v4 = validities

    log_likelihood = 0
    for t in range(len(choices)):
        option_A, option_B = optionAs[t], optionBs[t]
        value_A = np.array(option_A)
        value_B = np.array(option_B)
        scale_value_difference = temperature * np.sum(value_B - value_A)
        choice_probability_B = 1.0 /
            (1.0 + np.exp(-scale_value_difference))

        # compute log probability for actual choice
        p = choices[t] * np.log(choice_probability_B) +
            (1 - choices[t]) * np.log(1 - choice_probability_B)
        log_likelihood += p

    return -log_likelihood
```

Your function:

B.2 Learning

Learning prompt

This is a multi-armed bandit reinforcement learning task, where participants were asked to choose one of the two actions to gain monetary rewards. The task included two possible actions, and on each trial participants had to choose between the two.

They observed the reward for both their chosen and unchosen actions. Each action led to a reward according to an underlying probability distribution, which remained the same within a block but could change between blocks.

Here is a task data set from several participants:

Data from participant 1:

Block: 1, Trial: 1, Chosen action: 1, Reward for the chosen action: -1, Reward for the unchosen action: 1

Block: 1, Trial: 2, Chosen action: 1, Reward for the chosen action: -1, Reward for the unchosen action: -1

Block: 1, Trial: 3, Chosen action: 1, Reward for the chosen action: -1, Reward for the unchosen action: -1

...

Your task is to propose 3 unique cognitive models that could explain the observed behaviors in this dataset. When generating the models think in steps - for example: if on trial t participant chose a specific action and observed a given feedback for the chosen action AND the non-chosen action both, what is their subsequent action choice?

Ensure your models have distinct assumptions and parameter sets. Avoid repeating ideas used in previous iterations. Make sure all of the model parameters are actually being used.

Each model should be implemented as a Python function called 'cognitive_model1', 'cognitive_model2', and 'cognitive_model3'.

Each of the 3 functions should accept the following lists as arguments: actions, rewards, forgone rewards, and model parameters. Each function should return the negative log likelihood of observed actions given the model parameters.

When you write the function, also include the description of each parameter and what it does in the function's meta commented section.

Make sure the equations do not lead to nonsense values (e.g. watch out for division by 0). Make sure you write functions that are free of bugs, and can be executed. Here is an initial model guess of how participants solve the task:

```
def cognitive_model(actions, rewards, forgone_rewards, parameters):

    lr, neg_lr, inverse_temperature = parameters
    values = np.array([0.5, 0.5])
    log_likelihood = 0
    for t in range(len(actions)):
        p0 = np.exp(inverse_temperature*(values-values[actions[t]]))
        p = 1/(p0[0]+p0[1])

        log_likelihood += np.log(p)

        delta = rewards[t] - values[actions[t]]
        if delta >= 0:
            values[actions[t]] = values[actions[t]] + (lr * delta)
        else:
            values[actions[t]] = values[actions[t]] + (neg_lr * delta)

    return -log_likelihood
```

Your functions:

B.3 Planning

Planning prompt

In this task participants were instructed to obtain as many gold coins as possible by flying one of the two magic carpets (carpets carpets[0] and carpets[1]). After choosing one of the carpets, participants would fly to one of the two mountains (Pink or Blue) where they would encounter one of the two genies. Pink Mountain has genies genies[0] and genies[1], and Blue Mountain has genies genies[2] and genies[3].

Participants then chose one of the two encountered genies to ask for gold coins and was either given a gold coin or not. Different genies gave gold coins with varying probability. Therefore, this was a two-stage decision process: first, participant chose the flying carpet; second, after ending up on one of the mountains they chose the genie they would ask for gold coins.

In the task, the magic carpet carpets[0] generally flew to the Pink Mountain, and magic carpet carpets[1] generally flew to the Blue Mountain. However, on rare occasions a strong wind would send the magic carpet to a less likely destination (e.g., magic carpet carpets[0] would end up flying to the Blue Mountain). The participants might leverage their knowledge of the transition structure in the task (e.g., which carpet likely goes to which mountain) to maximize the chance of encountering the genie that on average yielded more gold).

Here is a task dataset from several participants:

Participant 1:

Trial 0: The participant chose magic carpet A and ended up on the Blue Mountain. The participant rubbed the lamp S and received 0 coins.

Trial 1: The participant chose magic carpet A and ended up on the Pink Mountain. The participant rubbed the lamp W and received 1 coin.

...

Your task is to propose 3 unique cognitive models that could explain the observed behaviors in this dataset. Think in steps - for example: if on trial t participant won a gold coin after choosing a magic carpet that took them to a mountain where they asked a genie for gold coins, what is their subsequent carpet choice? Do they consider how often they end up on that mountain after choosing that specific carpet?

Ensure your models have distinct assumptions and parameter sets. Avoid repeating ideas used in previous iterations. Make sure all of the model parameters are actually being used. Each model should be implemented as a Python function called 'cognitive_model1', 'cognitive_model2', and 'cognitive_model3'.

Each of the 3 functions should accept the following lists as arguments: action_1, state, action_2, reward, parameters. Each function should return the negative log likelihood of observed actions given its parameters.

When you write the function, also include the description of each parameter and what it does in the function's meta commented section. Note that for each parameter except the beta, the plausible bounds are between 0 and 1. Make sure the equations do not lead to nonsense values (e.g. watch out for division by 0).

Make sure you write functions that are free of bugs, and can be executed. Here is an initial model guess of how participants solve the task:

```
def cognitive_model(action_1, state, action_2, reward, parameters):

    learning_rate, beta = model_parameters
    n_trials = len(action_1)

    transition_matrix = np.array(
        [[.7, .3],
         [.3, .7]]
    )
    p_choice_1 = np.zeros(n_trials)
    p_choice_2 = np.zeros(n_trials)
    Q = np.zeros((3, 2))

    for trial in range(n_trials):
        max_q = np.max(Q[1:], axis=1)
        q_stage1 = transition_matrix @ max_q

        # Compute probability for choice 1
        exp_values = np.exp(beta * q_stage1)
        p_choice_1[trial] = np.exp(beta * q_stage1[action_1[trial]]) /
            np.sum(exp_values)

        # Compute probability for choice 2
        state_idx = state[trial] + 1 # Ensure correct state indexing
        exp_values_mf = np.exp(beta * Q[state_idx, :])
        p_choice_2[trial] = np.exp(beta * Q[state_idx, action_2[trial]]) /
            np.sum(exp_values_mf)

        delta = reward[trial] - Q[state_idx, action_2[trial]]
        Q[state_idx, action_2[trial]] += learning_rate * delta

    eps = 1e-10
    log_loss = -(np.sum(np.log(p_choice_1 + eps))
        + np.sum(np.log(p_choice_2 + eps)))

    return log_loss
```

Your functions:

B.4 Working memory

Working memory prompt

In this task participants are presented with a varying number of states (either 3 or 6), and asked to select one of the 3 possible actions. For each state there is a fixed correct action.

Following the action selection, participants observed feedback (0 or 1). Possible states reset at the start of each block (that is, there were no overlapping states between the blocks).

The objective of the experiment was to understand differences in learning across different cognitive load conditions (3 vs 6 state-action pairs).

Here is a task dataset from several participants:

Data from participant 1:

Block: 0, Set size:3, Trial: 0, State: 0, Chosen action: 0, Reward: 1
 Block: 0, Set size:3, Trial: 1, State: 1, Chosen action: 1, Reward: 1
 Block: 0, Set size:3, Trial: 2, State: 2, Chosen action: 2, Reward: 0
 ...

Ensure your models have distinct assumptions and parameter sets. Avoid repeating ideas used in previous iterations. Make sure all of the model parameters are actually being used. Each model should be implemented as a Python function called 'cognitive_model1', 'cognitive_model2', and 'cognitive_model3'. Each of the 3 functions should accept the following arrays as arguments: states, actions, rewards, blocks and model parameters. Each function should return the negative log likelihood of observed actions given its parameters.

When you write the function, also include the description of each parameter and what it does in the function's meta commented section. Note that for each parameter except the inverse temperature, the plausible bounds are between 0 and 1. Make sure the equations do not lead to nonsense values (e.g. watch out for division by 0).

Make sure you write functions that are free of bugs, and can be executed. Here is an initial model guess of how participants solve the task:

```
def cognitive_model(states, actions, rewards, blocks, parameters):
    lr, wm_weight, softmax_beta = parameters
    softmax_betawm = 50
    nA = 3
    nS = len(np.unique(block_states))
    q = (1 / nA) * np.ones((nS, nA))
    w = (1 / nA) * np.ones((nS, nA))
    w_0 = (1 / nA) * np.ones((nS, nA))
    log_p = 0
    for t in range(len(block_states)):
        a = block_actions[t]
        s = block_states[t]
        r = block_rewards[t]
        Q_s = q[s,:]
        W_s = w[s,:]
        p_rl = 1 / np.sum(np.exp(softmax_beta*(Q_s-Q_s[a])))
        p_wm = 1 / np.sum(np.exp(softmax_betawm * (W_s - W_s[a])))
        p_total = p_wm*wm_weight + (1-wm_weight)*p_rl
        log_p += np.log(p_total)
        delta = r-Q_s[a]
        q[s][a] += lr*delta
        if r > 0:
            w[s][a] = r
        else:
            w[s][a] += lr * (0-w[s][a])
    return -log_p
```

Your function:

C Decision making: Sanity checks using synthetic data

Task. We designed a task in which decision making agents chose between two options (A and B). Each option is characterized by three features, represented as integers ranging from 0 to 100.

Heuristics. We specifically examined two: the Take the Best and the Tallying heuristic. The Take the Best heuristic selects an option based solely on a single prioritized feature, ignoring comparisons across other features (Gigerenzer and Goldstein, 1996). Specifically, only the prioritized feature is

used to evaluate the two options, with the option that has the higher value for the prioritized feature winning in the comparison. The Tallying heuristic instead compares the two options based on all three features, counting the number of features for which one option has a higher value than the other, and favoring the option that has a higher number of superior features.

Synthetic data generation. We generated 80 decision problems (each problem including two sets of three features), in accordance with the task specification. During task generation, we ensured that the number of times option A or B is superior is balanced. For Take the Best simulations we deliberately prioritized the second feature to avoid the LLM making a potentially misleading assumption that the first feature should be prioritized. We simulated 40 decisions based on each of the two heuristics. Importantly, we initially ensured that our examples were unambiguous -avoiding cases where both heuristics would lead to the same decision. This approach guaranteed the identifiability of decision patterns unique to each heuristic. To further test the robustness of our approach, we simulated a second data set that introduced noise to the data simulation. We did this by increasing the proportion of decisions in which the final output of the heuristic was flipped, resulting in the opposite choice than the heuristic would actually make. We considered three noise levels: 0, 0.25, and 0.5. This progressively increased the level of confusion between the two heuristics (Figure 7B). At a noise level of 0.5, the decision patterns for both heuristics are indistinguishable and are equivalent to random guessing.

LLM prompting We queried the LLM to perform two tasks: (1) match the data to the source model (model identification) and (2) generate a cognitive model based on the observed data for both decision making and learning experiments (model generation). In the model identification task, the LLM prompt included the code for model candidates provided as Python strings along with simulated decisions. For the model generation task, the prompt included a description of the desired structure of the Python function (e.g., the function name, input arguments, and expected output). We considered three different prompting strategies: vanilla (containing only the description of the task setup), Step-Back (Zheng et al., 2023a), and Chain of Thought (CoT; Wei et al. 2022). This comparison aimed to identify the most effective prompting strategy for subsequent tests.

C.1 Results

Model identification. Model identification enabled us to evaluate the LLM’s ability to reason through decision making strategies and map the data to the underlying function. Consistent with previous results (Wei et al., 2022), CoT prompting led to the best LLM performance (Figure 7A); therefore, we used CoT prompting for all subsequent experiments. We found that in the noise-free data set, the LLM was perfect at identifying the source model based on the data (Figure 7A). When evaluated on the noisy data set, the LLM robustly identified the ground truth heuristic at noise level of 0.25 (Take the Best mean accuracy: 0.86 (SEM = 0.02); Tallying mean accuracy: 0.71 (SEM = 0.06)). At a noise value of 0.5 - corresponding to random guessing in the data generating process (Figure 7B) - the LLM’s heuristics predicted decisions at chance level (Take the Best accuracy > 0.5 test : $t(9)=1.80$, $p=0.10$; Tallying accuracy > 0.5: $t(9)=0.27$, $p=0.79$).

Model generation. Next, we tested whether the LLM could generate a decision making algorithm that aligned with the observed strategy patterns, simulated using either the Take the Best or Tallying heuristic. We evaluated the LLM-generated algorithms on unseen decision tasks, comparing their outputs to the predictions of the ground truth heuristic.

Our analysis of LLM-generated functions revealed that the LLM could successfully identify the two underlying heuristics: prioritizing a single feature for Take the Best simulations and performing across-feature comparisons for Tallying simulations (Figure 7C). The choices generated by the LLM-proposed models matched the ground truth heuristic choices with perfect accuracy in the evaluation tasks. It is notable, however, that for the Tallying heuristic there was a slight departure from the ground truth in the LLM-generated code – using the total sum of features instead of a tally of superior features. There are corner cases where these strategies would make diverging predictions (e.g., if the feature values are not normalized / in the same range). Nevertheless, the LLM proposed an equally valid alternative to the true data generating process.

To account for the noise in the noisy data set, the LLM-generated heuristics deviated more from the underlying heuristics. For noisy Take the Best data, the LLM still prioritized the second feature but

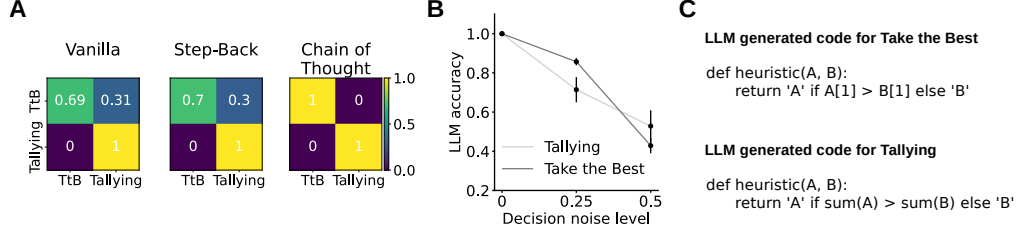


Figure 7: A) Identifying the source decision making heuristic by using the LLM to relate data to the source simulation code. We prompted the LLM to identify which of the two heuristics (Take the Best or Tallying) underlies the behavioral data from the two-alternative forced choice task, where the agent chooses between two options defined by three features. While exploring the LLM’s capacity to perform this task, we tried different prompting techniques. B) We tested the LLM with noisy decision making data, with injected noise increasing the confusion between the two heuristics. The LLM shows robustness to noise in the data - performance decreases proportionally with noise, but only reaches chance level when the heuristics are indeed indistinguishable (noise level of 0.5). Error bars represent standard error of the mean (SEM) across 10 runs. C) LLM-generated Python function heuristics closely align with the ground truth. The LLM-generated functions remained the same across 10 separate experiment runs.

modified the heuristic to apply only when a specific criterion (e.g., feature value differences above a certain threshold) was met. For noisy Tallying data, the LLM generated various strategies such as choosing based on the highest overall or minimum feature value.

D Learning: Sanity checks using synthetic data

Task. We implemented a two-armed bandit task, with each of the two options associated with a fixed probability of receiving a reward if selected (e.g. $p(r = 1|a_1) = 0.20$; $p(r = 1|a_2) = 0.80$). The rewards in our task were binary ($r \in \{0, 1\}$).

Learning models. The Rescorla-Wagner (RW) model (Rescorla, 1972) is commonly used to study learning dynamics in the bandit tasks. In the experiment, we considered the vanilla RW model and two variants of it - RW with two learning rates ($RW + \alpha^\pm$) and RW with stickiness ($RW + \kappa$).

The RW model posits that the value of each action (V) is determined by the history of rewards obtained from selecting that action. According to the RW model’s learning rule, the value of the selected action a (V^a) is updated on each trial t as follows:

$$V_{t+1}^a = V_t^a + \alpha \cdot (r - V_t^a)$$

where $r - V^a$ is the reward prediction error - a learning signal that drives the adjustment of the selected action value, and α represents a learning rate that captures the extent to which the action value is modified by the prediction error.

Learning models commonly rely on the softmax policy in conjunction with the RW learning rule, providing a way to transform action values into probabilities. The softmax policy introduces the exploration parameter β , which controls the degree to which action selection is deterministic:

$$P(a) = \frac{\exp(\beta \cdot V_t^a)}{\sum_{i=1}^N \exp(\beta \cdot V_t^i)}$$

The Rescorla-Wagner model with two learning rates ($RW + \alpha^\pm$) differentiates between outcomes that are better/worse than expected. More precisely, the model uses two distinct learning rates for action value updating, contingent on the valence of the prediction error:

$$V_{t+1}^a = \begin{cases} V_t^a + \alpha^+ (r - V_t^a) & \text{if } r - V_t^a \geq 0 \\ V_t^a + \alpha^- (r - V_t^a) & \text{if } r - V_t^a < 0 \end{cases}$$

The Rescorla-Wagner model with stickiness ($RW + \kappa$) has the same learning rule as the vanilla RW but its policy differs in that the additional weight κ is applied to the value of the action that was selected during the previous trial, resulting in a greater tendency to choose the previously selected action:

$$P(a) \propto \exp(\beta V + \kappa \mathbb{I}(a = a_{t-1}))$$

Synthetic data generation. To test how well we can recover the ground truth learning model, we simulated 100 agents from each of the two above-mentioned models on a two-armed bandit task, with each task comprising of 150 trials. The simulation parameters were randomly sampled for each agent in a range defined by plausible parameter bounds.

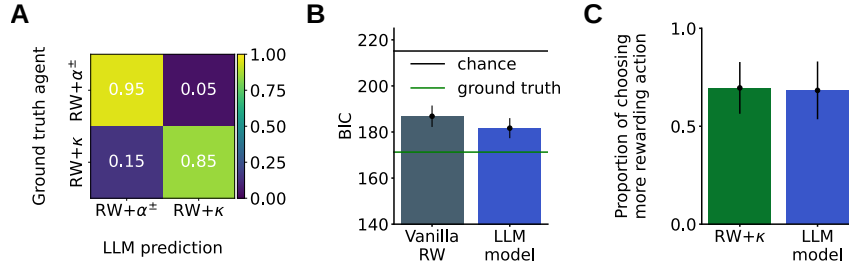


Figure 8: A) Model identification task. The LLM-generated 'ModelIdentification' function utilizes the SciPy differential evolution method to successfully differentiate between the two learning models. B) Evaluation of the LLM-generated cognitive model based on the data simulated from the $RW + \kappa$ revealed that it captured behavior better than the random guessing model and the vanilla RW. C) Simulation of the LLM-generated model showed that it captured the underlying propensity for choosing more rewarding actions. Error bars represent standard error of the mean (SEM) across simulated agents.

LLM Prompting. Identifying the source model by reasoning through the long sequences of learning data, which consisted of 150 actions and rewards, is much more challenging than identifying the underlying decision making heuristic. Additionally, differently configured models can produce similar action/reward trajectories - a common challenge in models of bandit tasks (Wilson and Collins, 2019). As a result, we modified the prompts we had developed for the decision making experiment.

Function generation for model identification. Unlike in the decision making case where the LLM directly returned the model identity, in the learning task the LLM instead generated a function for model identification. The function's arguments were predefined (e.g., lists of actions and rewards). The prompt encouraged the LLM to propose a method for matching the source model to the data without requiring step-by-step reasoning. The generated function was then manually evaluated to determine its accuracy in identifying the correct model.

Evaluation of LLM-generated cognitive models. The best LLM-generated cognitive models were evaluated in two steps at the end of the final sampling run. First, we compared the Bayesian Information Criterion (BIC; Watanabe 2013) of the best model to the ground truth (or the best model from the literature for human data), random and a competing model. Second, we manually implemented a simulation script based on the equations of the LLM-generated model. Using the best-fit parameters from the first step, the script simulated action choices according to the model's equations and parameters, with rewards determined by the probabilistic action-reward contingencies of the task. This step allowed us to assess how well the model captured behavioral patterns by comparing the simulated data with the ground truth.

Table 2: Examples of parameters in LLM-proposed cognitive models across various sampling runs. Modeling of these mechanisms is documented in previous research (Wilson and Collins, 2019).

MODEL PARAMETER	EXPLANATION
Decay	Forgetting mechanism (Paskewitz et al., 2022)
Random lapse	Random action-executions (Nassar and Frank, 2016)
Bias	Preference for a particular action (Balcarras et al., 2016)
Dynamic scaler	Parameter (e.g., learning rate) adjustment based on the trial number (Diederen and Schultz, 2015)
Exploration bonus	Directed exploration (Wilson et al., 2021)

D.1 Results

Model identification. We prompted the LLM to write a function, called `ModelIdentification`, that would identify the source model based on the underlying data in the learning experiments. We found that the LLM generated a function that performed an optimized search over possible model parameter values to find optimal log-likelihood, leveraging the differential evolution algorithm for optimization (Storn and Price, 1997) from the SciPy library. The proposed function returned the identity of the model associated with the smaller negative log-likelihood. We executed this LLM-generated function offline to determine the identity of the model. As shown in Figure 8A, we found that the $RW + \alpha^\pm$ model can be successfully identified 95% of times (SEM = 4) and the $RW + \kappa$ model could be identified about 85% of times (SEM = 7).

Model generation. For model generation, we run GeCCo for 10 iterations where the LLM generated three cognitive models that were fitted to the data offline on each run. The feedback was automatically constructed based on the model fits, and included as the part of the prompt in the subsequent iterations. Note that we only considered the best LLM-generated models across all runs for model fitting and comparison.

We found that the LLM recovered the $RW + \alpha^\pm$ model correctly from its simulated data. That is, the best generated model was the RW model with two learning rates, based on positive and negative feedback (ground truth model BIC: 78.78 (SEM = 5.3), LLM-generated model BIC: 78.40 (SEM = 1.3)).

For the $RW + \kappa$ model, the LLM did not discover the ground truth model. Instead, the best-fitting model contained a value-decaying mechanism, which lowered the value of the non-selected action on each trial. This can be viewed as a way to model the forgetting, or the information decaying mechanism, in the learning process. The LLM-generated model fitted better than the random guessing model (Figure 8B; $t(99) = 7.44, p < 0.001$) and the vanilla RW (Figure 8B; $t(99) = 2.36, p = 0.01$). As a sanity check, we also compared the data simulated based on the LLM-generated model to the ground truth by quantifying the proportion of trials in which the simulated agent selected the more rewarding option (Figure 8C). The results showed that the data simulated from the LLM-derived model approximated the ground truth data reasonably well (Figure 8B, proportion of selecting the more rewarding action: ground truth: 0.69 (SD = 0.13); LLM-generated model: 0.68 (SD = 0.13)).

Additionally, for both data sets, we checked which cognitive model parameters the LLM proposed across other sampling runs, beyond quantitatively studying only the best model. We found that the parameters were reasonable and among those commonly cited in the cognitive modeling literature.

E Human experiments

E.1 Decision making

In Hilbig and Moshagen (2014), 79 participants were instructed to repeatedly choose one of two options, labeled A and B, as shown in Figure 2A. They were told that these represent fictitious products and that they should infer which product is superior in terms of quality. In each trial, they were openly provided with the values of four binary features, explained to them as fictitious expert

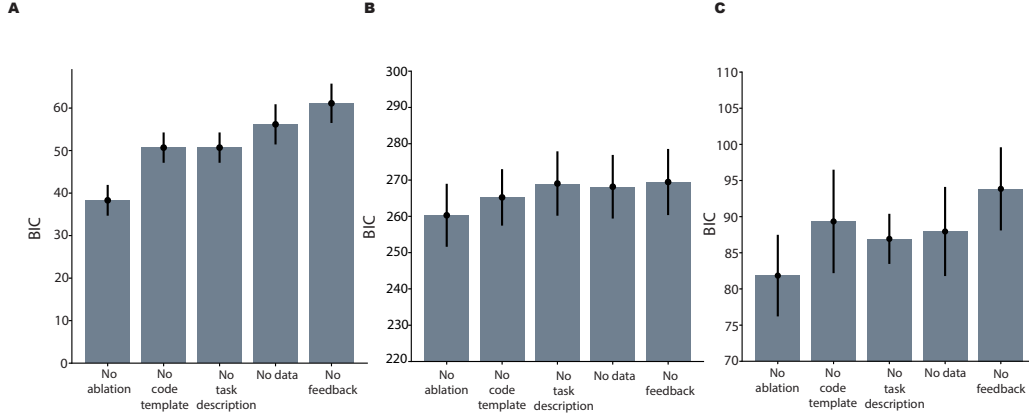


Figure 9: Ablation experiments. A) Ablation experiment in the decision making domain revealed that all components of the prompt were significant for contributing to performance of the LLM-generated models, with feedback removal from the prompt having the biggest effect. Results of the ablation experiment in the working memory domain B) and learning domain C) also support the impact of feedback.

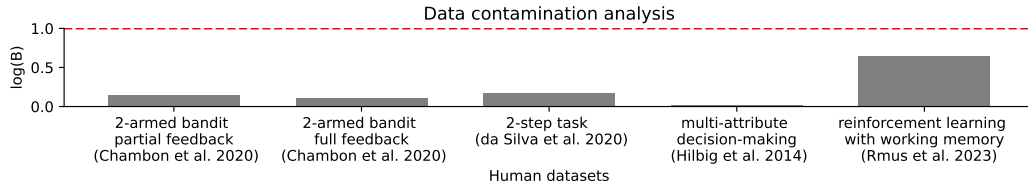


Figure 10: Data contamination analysis using the LogProber method (Yax et al. 2024). LogProber fits a two-parameter exponential model to the cumulative log-likelihood of each sequence being checked for contamination: $f(x) = -A(1 - \exp^{-Bx})$. Prompts that are memorized from the pretraining data will show a high acceleration ($\log B$). Following the results presented in (Yax et al., 2024), we set a threshold for possible contamination to $\log B \geq 1$. We considered prompts up to the point of the choices of human participants, which contained the task instruction, instruction given to help with code generation, and the code template, for the four human datasets on the Llama-3.1-70B-Instruct (base model for all winning LLM). The analysis revealed no evidence of contamination, with the acceleration term being substantially below 1 for all four human experiments investigated. Specifically, it was 0.1409 for partial feedback condition (Chambon et al., 2020), 0.1147 for full-feedback condition (Chambon et al., 2020), 0.0171 for multi-attribute decision-making (Hilbig and Moshagen, 2014), 0.1718 for two-step task (Feher da Silva and Hare, 2020), and 0.6481 for memory task (Rmus et al., 2023).

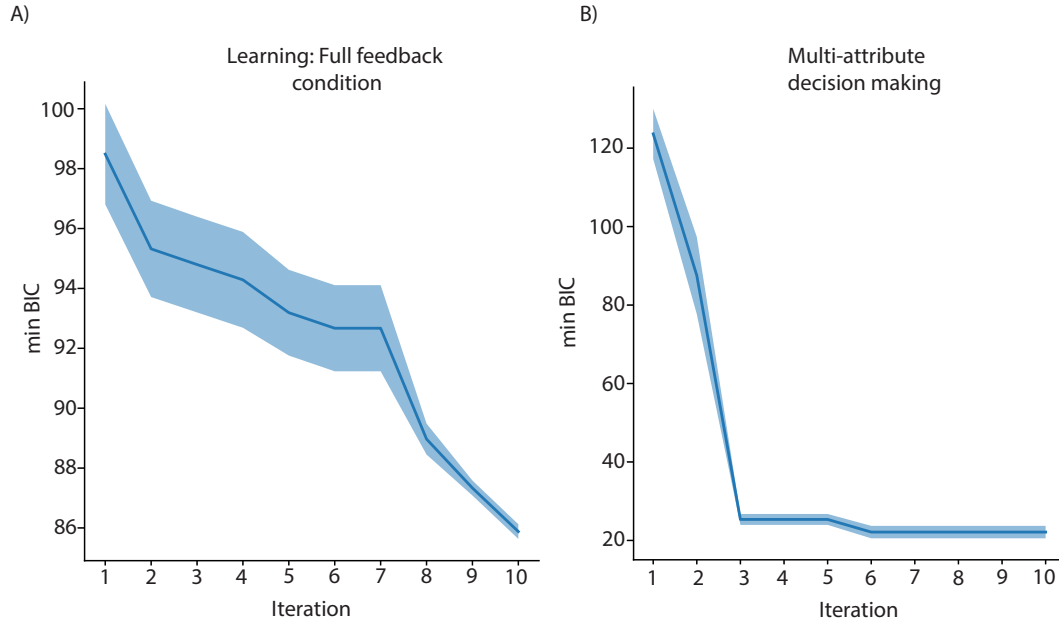


Figure 11: Minimum BIC scores decrease across sampling iterations. A) BIC improvement across iterations in the learning experiment. B) BIC improvement across iterations in the decision making experiment. Shaded area represents variability (SEM) across 5 different independent runs of GeCCo.

ratings. Furthermore, it was explained that the four ratings corresponded to ratings from four experts who differ in how well they typically predict product quality (i.e. feature validity), with their validities being .90, .80, .70, and .60, respectively. The participants performed 96 trials in total.

E.2 Learning

In the Chambon et al. (2020) study, 24 participants performed a two-armed bandit task designed to disentangle the effects of prediction-error valence on learning (see Appendix E for additional study details). The task consisted of 16 blocks. The action reward probabilities varied across blocks and were sampled from high-probability reward values (0.9, 0.6) or low-probability reward values (0.4, 0.1). The rewards in this task were binary, with ($r \in \{-1, 1\}$). In half of the blocks, the participants received feedback solely based on their selected action (partial feedback condition; see Figure 12A). In the other half, the participants received feedback based on their selected action, as well as counterfactual feedback from the alternative action they did not select (full feedback condition; see Figure 3A). We focused on the counterfactual condition, as it presents a broader space of possible data explanations (e.g. differences in learning based on actual and forgone rewards).

E.3 Planning

We used data from the “magic carpet” variant of the Daw two-step decision task, as implemented by Feher da Silva and Hare (2020). In this version, participants were told they were embarking on a series of magic carpet rides. On each trial, they first chose between two magic carpets (first-stage choice), each of which probabilistically transported them to one of two mountains (second-stage states). Each mountain housed two genies, one of which participants could select to potentially receive a reward (second-stage choice). The reward probabilities associated with each genie drifted slowly over time, encouraging ongoing exploration.

The task structure is designed to differentiate between model-free and model-based learning strategies. First-stage transitions are probabilistic: a chosen carpet leads to its associated mountain with 70% probability (common transition) and to the other mountain with 30% probability (rare transition). Second-stage (genie) choices are deterministic.

A model-free learner reinforces actions that previously led to reward, regardless of the transition type. In contrast, a model-based planner takes into account the transition structure: if a reward was obtained following a rare transition, the optimal strategy would be to switch first-stage choices on the next trial to more reliably reach the same second-stage state via a common transition.

E.4 Working memory

Participants were instructed to learn stimulus-response associations in order to earn as much reward as possible (Rmus et al., 2023). The number of stimulus-action associations varied across different task blocks (3 or 6), with new stimuli being introduced in each block. Participants encounter one stimulus at the time on the screen, pressed a corresponding key and were subsequently provided with deterministic feedback. The rewards in this task were binary, with $(r \in \{0, 1\})$. Each stimulus was shown nine times within a block and stimulus order was pseudo-randomly shuffled.

F Cognitive models

F.1 Decision making: Take The Best (TTB), Equal weighting (EQW), Weighted additive heuristic (WADD)

Heuristics are simple, resource-efficient approaches that individuals use to navigate the decision making process effectively (Tversky and Kahneman, 1974; Gigerenzer and Goldstein, 1996). Assume $\mathbf{x}_o = (x_{o1}, \dots, x_{oJ})$ to denote the feature vector for options $o \in \{A, B\}$, let $\mathbf{w} = (w_1, \dots, w_J)$ be the inferred feature weights and $\mathbf{v} = (v_1, \dots, v_J)$ be the feature validities, where J is total number of features. As potential models (Gigerenzer and Goldstein, 1996), Hilbig and colleagues considered three heuristics apart from the winning model (pWADD), namely, Take the Best (TTB), Equal Weighting (EQW), and Weighted Additive (WADD), to explain human choices. The Take the Best (TTB) heuristic selects an option based solely on a single prioritized feature, ignoring comparisons between other features (Gigerenzer and Goldstein, 1996). EQW heuristic instead compares options based on all four features, favoring the option that has a higher sum of superior features. WADD heuristic is similar to EQW, but weighs the four features differently as per the provided feature validities. Finally, pWADD is the probabilistic variant of the WADD heuristic that uses inferred feature weights instead of using the feature validities (note β is the temperature term). The equations for each of the models can be found in the following.

$$\begin{aligned}
\text{TTB}(A, B) &= \arg \max_{o \in \{A, B\}} x_{oj^\dagger}, & j^\dagger &= \arg \max_{j \in \{1, 2, \dots, J\}} v_j \\
\text{EQW}(A, B) &= \begin{cases} A, & \sum_{j=1}^J (x_{Aj} - x_{Bj}) > 0, \\ B, & \text{otherwise.} \end{cases} \\
\text{WADD}(A, B) &= \begin{cases} A, & \sum_{j=1}^J v_j (x_{Aj} - x_{Bj}) > 0, \\ B, & \text{otherwise.} \end{cases} \\
\text{pWADD}(A, B) &= \begin{cases} p(A), & \frac{1}{1 + \exp[-\beta \sum_{j=1}^J w_j (x_{Aj} - x_{Bj})]}, \\ p(B), & 1 - p(A). \end{cases}
\end{aligned} \tag{1}$$

F.2 Learning: Rescorla Wagner model with 4 learning rates, softmax policy and perseveration

The winning model from Chambon et al. 2020 is a variant of the RW model with four learning rates ($\alpha^{c+}, \alpha^{c-}, \alpha^{u+}, \alpha^{u-}$, $c = \text{chosen}, u = \text{unchosen}$). Thus, the model updates values of actions

differently, depending on whether the outcome of the action is better/worse than expected, as well as whether the action was chosen by the participant or not:

$$V_{t+1}^a = \begin{cases} V_t^a + \alpha^{c+} (r - V_t^a) & \text{if } r - V_t^a \geq 0 \\ V_t^a + \alpha^{c-} (r - V_t^a) & \text{if } r - V_t^a < 0 \end{cases} \quad V_{t+1}^{1-a} = \begin{cases} V_t^{1-a} + \alpha^{u+} (r' - V_t^{1-a}) & \text{if } r' - V_t^{1-a} \geq 0 \\ V_t^{1-a} + \alpha^{u-} (r' - V_t^{1-a}) & \text{if } r' - V_t^{1-a} < 0 \end{cases}$$

where $r - V_t^a$ is the reward prediction error (RPE) for the chosen action, and $r' - V_t^{1-a}$ is the RPE for the unchosen action based on the forgone reward. RPEs drive value updates, with α^{c+} and α^{u+} denoting learning rates for positive prediction errors of chosen and unchosen actions, respectively, and α^{c-} and α^{u-} for negative prediction errors.

The softmax policy introduces the exploration parameter β , which controls the degree to which action selection is deterministic:

$$P(a) = \text{softmax}(a) = \frac{\exp(\beta \cdot V_t^a)}{\sum_{i=1}^N \exp(\beta \cdot V_t^i)}$$

We have also included the perseveration parameter κ which assigns higher value to the action selected on the previous trial and thus captures the human tendency to repeat previously selected actions: $P(a) \propto \exp(\beta V + \kappa \mathbb{I}(a = a_{t-1}))$.

E.3 Planning: The Model-based/Model-free Hybrid model

The value of the selected first-stage action a_1 in state s_1 is updated based on the reward prediction errors from both the first and second stages, as follows:

$$Q_{t+1}^{\text{MF}}(s_1, a_1) = Q_t^{\text{MF}}(s_1, a_1) + \alpha_1 [Q_t^{\text{MF}}(s_2, a_2) - Q_t^{\text{MF}}(s_1, a_1)] + \alpha_1 \lambda [r_t - Q_t^{\text{MF}}(s_2, a_2)]$$

Model-free values of the second-stage action a_2 performed in the second-stage state s_2 are updated at the end of the trial (e.g., when the reward is observed) based on the reward prediction error—defined as the difference between the received reward r_t and the current value of the chosen action:

$$Q_{t+1}^{\text{MF}}(s_2, a_2) = Q_t^{\text{MF}}(s_2, a_2) + \alpha_2 [r_t - Q_t^{\text{MF}}(s_2, a_2)]$$

The model-based and model-free value of second-stage actions in state s_2 are equal:

$$Q_t^{\text{MB}}(s_2, a_2) = Q_t^{\text{MF}}(s_2, a_2)$$

The model-based value of each first-stage action is computed at the time of decision-making based on the values of the second-stage actions, as follows:

$$Q_t^{\text{MB}}(s_1, a_1) = \sum_{s_2 \in \mathcal{S}} P(s_2 | s_1, a_1) \max_{a_2 \in \mathcal{A}} Q_t^{\text{MB}}(s_2, a_2)$$

First-stage choices are determined by combining model-free and model-based value estimates for each state–action pair. These are integrated using a weighting parameter w (where $0 \leq w \leq 1$), which reflects the relative influence of model-based planning. The resulting action values are then passed through a softmax function to generate choice probabilities:

$$P_t(s_1, a_1) = \frac{e^{\beta_1 [w Q_t^{\text{MB}}(s_1, a_1) + (1-w) Q_t^{\text{MF}}(s_1, a_1) + p \times \text{rep}_t(a_1)]}}{\sum_{a' \in \mathcal{A}} e^{\beta_1 [w Q_t^{\text{MB}}(s_1, a') + (1-w) Q_t^{\text{MF}}(s_1, a') + p \times \text{rep}_t(a')]}}$$

$$P_t(s_2, a_2) = \frac{e^{\beta_2 Q_t(s_2, a_2)}}{\sum_{a' \in \mathcal{A}} e^{\beta_2 Q_t(s_2, a')}}}$$

F.4 Working memory: The Reinforcement Learning - Working Memory model

The learning dynamics in the RL module of this model are the same as in the RW model, see Appendix F.2.

$$\begin{aligned}\delta_{\text{RL}} &= r - Q_t(s, a) \\ Q_{t+1}(s, a) &= \begin{cases} Q_t(s, a) + \alpha^+ \delta_{\text{RL}} & \text{if } \delta_{\text{RL}} > 0 \\ Q_t(s, a) + \alpha^- \delta_{\text{RL}} & \text{if } \delta_{\text{RL}} \leq 0 \end{cases}\end{aligned}$$

where Q denotes the action value, s the presented stimulus and a the chosen action.

The WM module, rather than integrating information over time, memorizes information from the previous trial. It also allows to encode WM stimulus action weights differently for positive and negative reward prediction error. In the case of positive reward prediction error,

$$\begin{aligned}W_{t+1}(s, a) &= r & \text{if } \delta_{\text{WM}} > 0 \\ W_{t+1}(s, a) &= W_t(s, a) + v \delta_{\text{WM}} & \text{if } \delta_{\text{WM}} \leq 0\end{aligned}$$

where v can be seen as the strength of the imperfection of the association and is identical to the relative neglect of negative feedback from the RL module $\frac{\alpha^-}{\alpha^+}$. Additionally, to model forgetting of the stimulus-action associations that are not seen in a current trial, the WM module has a decay parameter ϕ which pulls the WM weights to their initial values W_0 .

$$W_{t+1}(s_i, a_j) = W_t(s_i, a_j) + \phi (W_0(s_i, a_j) - W_t(s_i, a_j)) \quad \forall s_i \forall a_j \neq (s, a)$$

To make choices, the Q -values and WM weights are transformed into choice probabilities using a softmax policy:

$$\begin{aligned}P_{\text{RL}}(a|s) &= \frac{\exp(\beta^{\text{RL}} Q_t(s, a))}{\sum_{i=1}^{n_A} \exp(\beta^{\text{RL}} Q_t(s, a_i))} \\ P_{\text{WM}}(a|s) &= \frac{\exp(\beta^{\text{WM}} W_t(s, a))}{\sum_{i=1}^{n_A} \exp(\beta^{\text{WM}} W_t(s, a_i))}\end{aligned}$$

The RL and WM policies are then combined using a WM weight parameter ω , determining the relative reliance on WM in taking an action.

$$P_{\text{RL-WM}}(a|s) = \omega^{n_s} P_{\text{WM}}(a|s) + (1 - \omega^{n_s}) P_{\text{RL}}(a|s)$$

ω is dependent on the set size of stimulus-response association pairs n_s , because as WM's capacity is exceeded with increasing set size, individuals should rely more on RL. The model thus predicts learning to be fast when set size is small due to high reliance on WM and learning to be more incremental when set size is high due to increased reliance on RL. Finally, the policy also captures value-independent random lapses:

$$P = (1 - \epsilon) P_{\text{RLWM}} + \epsilon \frac{1}{n_A}$$

where $\frac{1}{n_A}$ denotes the uniform random policy and ϵ the noise parameter.

G Partial feedback (learning) experiment

As a part of the learning experiment we considered two datasets by Chambon et al. (2020) - learning in partial feedback condition and full feedback condition. In the partial feedback condition participants only observed the outcome of the actions they chose (as opposed to also observing forgone rewards

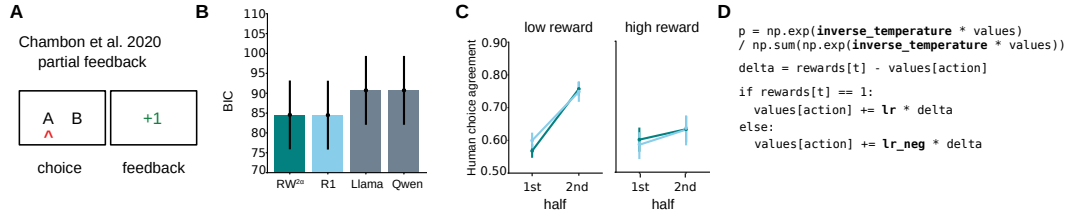


Figure 12: Learning experiment: partial feedback A) Schematic of the learning task from Chambon et al. (2020), where participants chose between two options and received feedback only for the option they chose. B) Model fit comparison: LLM-generated model from R1 on average fit as well as $RW^{2\alpha}$. C) Posterior predictive checks showing close alignment between human data and predictions of the best LLM model. D) Code of the top LLM-generated model (R1), differentiating learning based on positive and negative feedback.

based on unchosen actions, Fig. 12A). Chambon et al. (2020) reported a two-learning rate RW model ($RW^{2\alpha}$) as the best model in their experiment. We applied the GeCCo to the partial feedback dataset using a vanilla RW model (with a learning rate and an inverse softmax temperature) as the function template. We found that the R1 proposed the best cognitive model (Fig. 12B) which closely aligned with $RW^{2\alpha}$ in terms of containing two learning rates, with a notable difference that the learning rates differentiated learning based on positive/negative feedback, rather than the prediction error valence; Fig. 12D. This model fits neatly into current debates about learning from valenced rewards, providing a valid alternative hypothesis to differentiating learning rate based on valence of reward prediction error. Posterior predictive checks revealed close agreement between the LLM-generated model and human data; Fig. 12C.

H Posterior predictive checks

Posterior predictive checks are a crucial tool for evaluating a model’s generative validity—that is, its ability to reproduce patterns observed in real behavior—thus offering insight into how well the model captures the underlying cognitive processes. To perform posterior predictive checks, we used ChatGPT to convert the LLM-generated model-fitting functions—originally designed to take participant behavior and parameter values as input and return negative log likelihood—into simulation functions that generate behavior from parameter values based on the model equations. Using the best-fitting parameters for each participant, we simulated behavior and compared it to the actual human data through domain-specific, informative analyses applied to both simulated and real datasets.

In the decision making experiment, we computed the proportion of option choices that are explained by three different heuristics: Equal Weighting (EQW), Take The Best (TTB), and Weighted Additive (WADD); see section F.1 for details about the heuristics. We conducted this analysis on 1) human data, 2) data simulated from the best LLM-generated model, and 3) data simulated from pWADD - the best performing model from the literature. We then examined correlation between the heuristic-specific proportions on a participant-level, as the data was simulated using the actual decision tasks (e.g. options and features) from the experiment participants experienced and best-fit model parameters for each participant.

In the learning experiment, we simulated data for each participant using best fitting parameters, and block-specific reward probability contingencies specified in the Chambon et al. (2020) (high or low). We computed alignment with human choices for both the best performing LLM model, and $RW^{4\alpha}$ in high/low reward blocks separately, as well as for early/late trials in blocks to test whether the models capture temporal learning dynamics.

In the planning experiment we performed the canonical analysis from Daw et al. (2011), which looks at the likelihood of participants repeating the same stage 1 response, contingent on whether 1) participants received reward at the end of the previous trial and 2) experienced a common or a rare transition en route to feedback.

In the RL-WM experiment, we computed participant-specific learning curves showing the proportion of correct choices as a function of the stimulus iterations - the number of times participants en-

countered the stimulus. We used participant-specific stimulus sequences and correct stimulus-action mappings, to ensure that the behavior between simulated and real data is comparable.

I Using AIC for the best model selection in the GeCCo pipeline

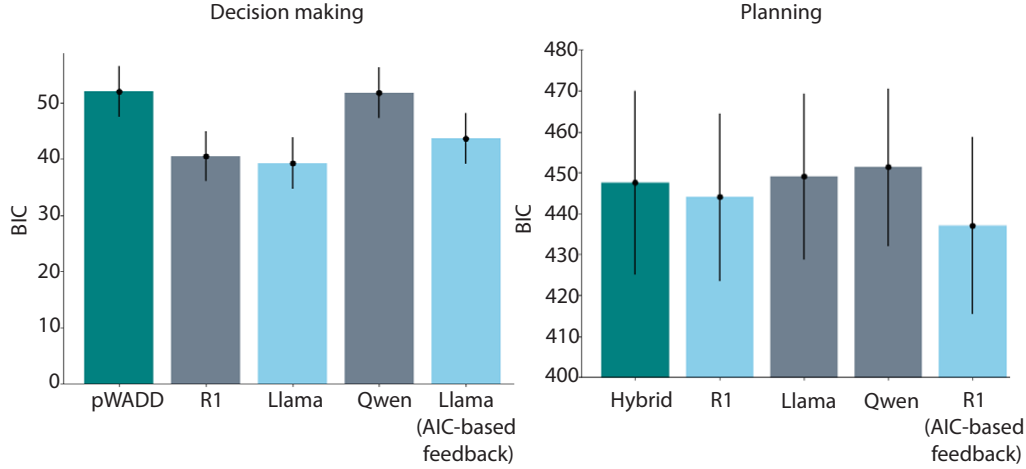


Figure 13: Comparison of the best GeCCo-generated models in the decision making and planning domains, including models from a pipeline variant using AIC-based feedback. This demonstrates that the pipeline is adaptable to different fit metrics and that AIC-based results closely replicate those obtained with BIC-based feedback.

J Using LLM-generated template in the prompt

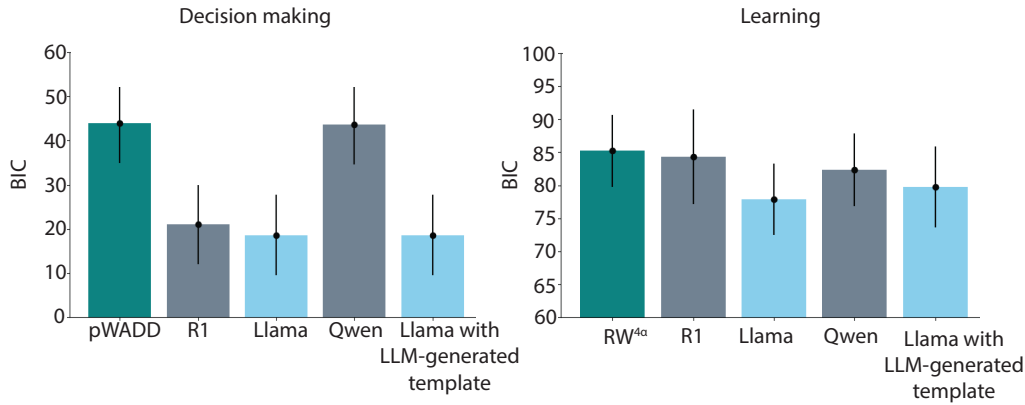


Figure 14: Comparison of the best GeCCo-generated models in the decision-making and learning domains, including models from a pipeline variant using an LLM-generated (rather than hand-crafted) template. The results show performance comparable to using a hand-crafted template.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [NA].
- [NA] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for NA).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will be asked to also include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While "[Yes]" is generally preferable to "[No]", it is perfectly acceptable to answer "[No]" provided a proper justification is given (e.g., "error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering "[No]" or "[NA]" is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist",**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: The abstract and the introduction reflect paper's contribution and scope: using three Large Language Models (R1-Distilled, Llama, Qwen) to generate computational cognitive models in four cognitive domains (Learning, Decision making, Planning and Memory), with LLM-generated models matching or surpassing the best handcrafted models reported in literature. These claims match results reported in the figures and results section of experiments 1-4. Additionally, we reference the control results in the abstract (performing miscellaneous analyses to assert robustness of our results), which are reported in the control experiment results section.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.

- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We discuss limitations including our approach being applied to only four cognitive domains that are not reflective of the entire field of cognitive science, potential susceptibility of LLMs to prompt formulation, limited exploration of feedback options, etc.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[NA\]](#)

Justification: Our work is not theoretical, therefore this does not apply.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We provide 1) the prompt structure that generalized across the domains we tested, as well as fully detailed prompts we used for each experiment 2) details on LLMs we used to run experiments and which parameters we set (e.g., temperature), 3) model fitting instructions (e.g., which optimizer we used and for how many random starting points), 4) examples of text version of different data sets. All the data and the code are available in online repositories. We will make them available before the conference.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All the data and the code are available in online repositories. We will make them available before the conference.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.

- While we encourage the release of code and data, we understand that this might not be possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyper-parameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: We explain that for each domain we used three separate parts of the data set for constructing the LLM prompt, evaluating the LLM-generated models during sampling, and comparing the best LLM-generated models to the baselines. We will make the data and the code used for splitting train/evaluation/test sets available before the conference.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We perform appropriate statistical tests for our results, and also explain the meaning of the error bars (e.g. whether it's across the runs or participants in the data set)

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.

- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We specify in the Methods section that for each of our experiments running an experiment took a maximum of eight hours on four Nvidia A100s with 40GB memory each.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research conducted in the paper conforms with the NeurIPS Code of Ethics.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [NA]

Justification: Our research is foundational and not tied to particular applications or deployments.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to

generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: Paper poses no such risks.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We cited all original papers that produced the datasets we used.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [NA]

Justification: We did not collect new data for this paper, and all of our experiments are done in-context (i.e. without model training).

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: We used data collected by other researchers for their original publications, and were not involved in the data collection protocol. We included the description of the tasks provided to the participants, in as much detail as is described in the original papers.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: We did not collect any new human data for our experiments.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Since LLMs are at the core of our experiments, we described in detail which LLMs we used (including what family, size, and how they were prompted).

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (<https://neurips.cc/Conferences/2025/LLM>) for what should or should not be described.